

My Notes: Recursion

The professor's working notes, not a study guide

A word before you read. These are my notes from years of teaching this unit, cleaned up so they are readable. While C/C++ was my first language, I am a bit rusty because I have taught Java for years now. For these notes, I have been using AI to help type these up, check for issues, and fill in explanations, which means a wrong or imprecise statement can slip in and survive longer than it should, and I do not fully trust the AI, so I am still editing. They are **not** meant to be a study guide, they are not gospel, and not everything here shows up in a lecture (and some lecture items are not here). If you find something that looks wrong, contradicts the book, or just does not behave the way I claim when you run it, **tell me**. You will be doing students and me a favor, and you will understand it better for having caught it. The textbook is the tiebreaker, and the compiler beats us both. We are a two-language course, thus I will do my best to show everything in Java (**CIS**) and C/C++ (**CS**).

1. What recursion actually is

A **recursive** method solves a problem by calling itself on a *smaller* version of the same problem. That is the whole definition. Everything else is bookkeeping.

The mental picture I keep coming back to is the mirror facing a mirror: each reflection is a smaller copy of the same scene, nested inside the last, until they get too small to see. A recursive call is that smaller reflection, the same method, the same shape of problem, one step reduced.

Recursion is not iteration. A loop repeats a block of code while a condition holds. Recursion re-enters the *same method* with new arguments, and each entry gets its own private set of local variables (this matters; hold that thought for §3). Anything you can write one way you can, in principle, write the other. The question is never “can I?” It is “which one makes this problem read clearly and run acceptably?”

Why bother, then? Because some problems are *defined* in terms of themselves: a folder contains files and folders; a family tree is people whose parents are people; the answer for n is n times the answer for $n - 1$. When the problem has that shape, recursion lets the code match the shape, and code that matches its problem is code you can actually reason about.

2. The three parts of every recursive solution

Every correct recursion has exactly these three pieces. If you can find all three in your code, it will probably work; if one is missing or wrong, it will not.

1. **Base case:** a version of the problem small enough to answer *directly*, with no further recursive call. This is the floor. Every chain of calls has to land on one.
2. **Recursive step:** solve a *smaller* instance of the same problem, one that is strictly closer to the base case. “Smaller” is doing real work here: if the argument does not move toward the base case, you never stop.
3. **The return (backtrack) path:** each call returns its answer to the call that made it, and

those answers combine as the recursion backtracks.

The framing I use in class: a recursive method is a **promise**. `factorial(n)` promises: “if you can get me the answer for `n - 1`, I will turn it into the answer for `n`.” Every call makes that promise to a smaller call. The base case is where the promises finally get paid, when someone answers without passing the buck, and then the answers backtrack up through the waiting calls.

```
// Java
long factorial(int n) {
    if (n <= 1) return 1;           // base case: pays off directly
    return n * factorial(n - 1);    // recursive step: smaller n, then combine
}
```

```
// C++
long factorial(int n) {
    if (n <= 1) return 1;           // base case
    return n * factorial(n - 1);    // recursive step
}
```

Yes, for this one the two are line for line identical. Recursion is a control-flow idea, not a language feature. The languages start to diverge only when data structures get involved (§9).

A base case you cannot reach is the same as having no base case. The line `if (n == 0) return 1;` looks fine until someone calls `factorial(-3)`; `n` marches off toward negative infinity and never hits 0. Writing `n <= 1` is safer here precisely because it is a *floor*, not a single point. Prefer base conditions that catch everything at or past the boundary.

3. How it actually runs: the call stack and the box trace

This is the section that makes recursion stop feeling like magic.

Every time a method is called, recursive or not, the machine sets aside a small block of memory called a **stack frame** (an “activation record” in the book). The frame holds that call’s parameters, its local variables, and a note about where to return when it finishes. Frames stack up; the newest is on top. When a call returns, its frame is thrown away and control drops back to the frame beneath it.

So the reason each recursive call has its own private `n` is that each call has its own *frame*. There is no shared `n` being changed: there are several different values of `n`, one per frame, and they quietly disappear in reverse order as the calls return.

The **box trace** is just drawing those frames as boxes as they come and go. Trace `factorial(3)` by hand at least once in your life:

```
call   factorial(3)  -> 3 * factorial(2)    (waiting on factorial(2))
call   factorial(2)  -> 2 * factorial(1)    (waiting on factorial(1))
call   factorial(1)  -> 1                    base case, no more calls
return factorial(1) = 1
return factorial(2) = 2 * 1 = 2
return factorial(3) = 3 * 2 = 6
```

Two directions matter, and students blur them together: the **calls go down** (each frame spawns a smaller one) and the **frames backtrack up** (each one finishes the multiply it was waiting on). Nothing above the base case can finish until the base case answers; every frame is parked mid-expression, holding a $3 \times$ or $2 \times$ it cannot complete yet. On the slides I draw a gold arrow going down for the calls and a cyan arrow coming up for the backtrack; do the same on your paper and the “when does the multiply happen?” confusion goes away.

Draw the trace before you trust the code. If your hand trace and your program disagree, one of them has a bug and it is worth finding out which. That is how you learn where your mental model is wrong.

4. Two shapes: linear versus tree recursion

Recursion problems come in shapes, and telling them apart tells you a lot about cost.

Linear recursion makes *one* recursive call per step. The calls form a straight chain: `fact(4) -> fact(3) -> fact(2) -> fact(1)`. Depth grows with `n`, work grows with `n`. Cheap, predictable, and often trivially convertible to a loop.

Tree recursion makes *two or more* recursive calls per step, so the calls branch out into a tree. The classic is Fibonacci:

```
// Java -- correct, and a cautionary tale
long fib(int n) {
    if (n <= 1) return n;           // fib(0)=0, fib(1)=1
    return fib(n - 1) + fib(n - 2); // TWO calls -> branches
}
```

```
// C++
long fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

Draw the **recursion tree** for `fib(5)` and you will see the problem immediately: `fib(3)` gets computed twice, `fib(2)` three times, and it gets exponentially worse as `n` grows. The plain recursive Fibonacci is *correct* and *beautiful* and *terrible*: it recomputes the same subproblems over and over. This is the standard motivating example for **memoization** (cache each answer the first time) and eventually **dynamic programming**. It is also the cleanest proof I know that recursion is not automatically efficient. Elegant and fast are different axes.

If you want the historical footnote: the sequence shows up in Fibonacci’s thirteenth-century rabbit-breeding puzzle, and it keeps turning up in pinecones, sunflower spirals, and every data-structures course ever taught. If you have watched *Star Trek*, tribble populations grow the same way: each step depends on the steps before it, and it explodes fast.

5. Head recursion versus tail recursion

This is a different way to slice recursion, orthogonal to linear versus tree. It asks a narrower question: where does the recursive call sit inside the method?

Head recursion: the recursive call happens first, before any other work in that call. Everything else the method does happens after the call returns, on the backtrack (§3). The countdown example makes this concrete: recurse before printing, and the printing only happens as the calls backtrack.

```
void headExample(int n) {
    if (n == 0) return;
    headExample(n - 1);           // recurse first
    System.out.println(n);       // work happens after, on the backtrack
}
```

Tail recursion: the recursive call is the last action the method performs, with nothing left pending when it returns. Our factorial as written earlier (§2) is not tail recursive, since `return n * factorial(n - 1);` still has a multiplication waiting on the call to come back. Making it tail recursive means carrying the running product along as an extra parameter, so the work happens before the call instead of after:

```
long factorialTail(int n, long acc) {
    if (n <= 1) return acc;
    return factorialTail(n - 1, n * acc); // last action, nothing pending
}
```

```
long factorialTail(int n, long acc) {
    if (n <= 1) return acc;
    return factorialTail(n - 1, n * acc);
}
```

Why this matters: a tail call carries no pending work, so a compiler that recognizes the pattern can reuse the current stack frame instead of stacking a new one, turning the recursion into a loop under the hood. This connects to the tail-call point made under C++ versus Java (§9): do not rely on that optimization happening, since neither the JVM nor C++ compilers guarantee it. Write the recursion for clarity, and treat any frame savings as a bonus.

One trap worth naming: seeding the accumulator wrong (`factorialTail(n, 0)`) silently returns zero for everything, and the extra parameter changes the method’s public signature, often reason enough to leave a clean recursive definition alone unless stack depth is an actual problem.

6. A small gallery of classic recursions

Read these for the *shape*, not to memorize them. Each is short on purpose.

Sum of 1 to n

Linear, the “hello world” after factorial.

```
int sumTo(int n) { return (n == 0) ? 0 : n + sumTo(n - 1); }
```

```
int sumTo(int n) { return (n == 0) ? 0 : n + sumTo(n - 1); }
```

Power, x^n

Linear.

```
double power(double x, int n) { return (n == 0) ? 1 : x * power(x, n - 1); }
```

```
double power(double x, int n) { return (n == 0) ? 1 : x * power(x, n - 1); }
```

Bonus worth knowing: fast exponentiation halves n each step. Square the result of `power(x, n/2)`, then multiply by an extra x if n is odd, turning n multiplications into about $\log_2 n$. Same idea as binary search: throw away half the work each call.

Greatest common divisor

Euclid's method. The base case is the interesting part.

```
int gcd(int a, int b) { return (b == 0) ? a : gcd(b, a % b); }
```

```
int gcd(int a, int b) { return (b == 0) ? a : gcd(b, a % b); }
```

Reverse an array in place

Recursion over a shrinking range, which connects to your arrays background.

```
// Java
void reverse(int[] a, int lo, int hi) {
    if (lo >= hi) return;           // base: 0 or 1 element left
    int t = a[lo]; a[lo] = a[hi]; a[hi] = t;
    reverse(a, lo + 1, hi - 1);    // smaller range
}
```

```
// C++ -- pass the vector by reference so we edit the original, not a copy
void reverse(std::vector<int>& a, int lo, int hi) {
    if (lo >= hi) return;
    std::swap(a[lo], a[hi]);
    reverse(a, lo + 1, hi - 1);
}
```

That `&` in the C++ version is not decoration. See §9.

Binary search

Divide and conquer. This is the “guess a number from 1 to 10” game from class, and it is in the book as the k -th-smallest cousin.

```
// Java -- returns index of key, or -1
int bsearch(int[] a, int key, int lo, int hi) {
```

```

    if (lo > hi) return -1; // base: empty range, not found
    int mid = lo + (hi - lo) / 2; // avoids overflow vs (lo+hi)/2
    if (a[mid] == key) return mid; // base: found it
    if (key < a[mid]) return bsearch(a, key, lo, mid - 1); // search left half
    else return bsearch(a, key, mid + 1, hi); // search right half
}

```

```

// C++
int bsearch(const std::vector<int>& a, int key, int lo, int hi) {
    if (lo > hi) return -1;
    int mid = lo + (hi - lo) / 2;
    if (a[mid] == key) return mid;
    if (key < a[mid]) return bsearch(a, key, lo, mid - 1);
    else return bsearch(a, key, mid + 1, hi);
}

```

Two base cases here, which is normal and fine: “found it” and “ran out of range.” Note the array must be **sorted** for this to mean anything, the same caution as `Arrays.binarySearch`.

Towers of Hanoi

The one everybody remembers. Genuine tree-shaped recursion, and the code is almost embarrassingly short for how clever it feels.

```

// Java -- move n disks from 'from' to 'to' using 'via'
void hanoi(int n, char from, char to, char via) {
    if (n == 0) return; // base: nothing to move
    hanoi(n - 1, from, via, to); // move the top n-1 out of the way
    System.out.println(from + " -> " + to);
    hanoi(n - 1, via, to, from); // move them back on top
}

```

```

// C++
void hanoi(int n, char from, char to, char via) {
    if (n == 0) return;
    hanoi(n - 1, from, via, to);
    std::cout << from << " -> " << to << "\n";
    hanoi(n - 1, via, to, from);
}

```

Moving n disks takes $2^n - 1$ moves. The legend has monks moving 64 disks to end the world; at one move a second that is about 585 billion years, so we are probably safe. This is a good one to trace for $n = 3$ and then just *believe* for larger n . Trusting the recursion instead of unrolling it in your head is a skill worth practicing here.

7. Recursion versus iteration: choosing

They are interchangeable in power, so choose on *clarity first, cost second*.

Reach for recursion when the problem is naturally self-similar (trees, nested folders, grids, or “the answer depends on the same answer for a smaller input”) and the recursive version is genuinely easier to read than the loop.

Reach for a loop when it reads just as clearly and you care about speed or memory. A simple countdown or running sum is a loop, full stop. And when recursion would go very deep (tens of thousands of calls), a loop avoids the stack blowing up (§8).

Every linear recursion has a straightforward loop equivalent; that is often the pragmatic rewrite. Tree recursions (Hanoi, many tree and graph walks) resist tidy loop rewrites: you end up managing an explicit stack by hand, which is just recursion with extra steps and more bugs. When the recursion is doing something a loop cannot do cleanly, keep the recursion.

Rule of thumb I actually use: **write it recursively if that is how you understand the problem, and convert to a loop only if a profiler or a stack overflow makes you.**

8. The traps everyone hits once

No base case, or an unreachable one. The method never stops calling itself, the stack fills, and you get a `StackOverflowError` (Java) or a crash or segfault (C++). This is the number-one recursion bug. Before you run anything, ask: what is my base case, and does every path reach it?

A recursive step that does not shrink. A call to `factorial(n)` from inside `factorial(n)`, or `factorial(n - 0)`, or forgetting to change the argument, is infinite recursion wearing a disguise. Each call must move *strictly* toward the base case.

Off by one in the base condition. The conditions `n == 1`, `n <= 1`, and `n == 0` are not interchangeable, and the wrong one either misses inputs (negatives sail past `== 1`) or returns a wrong answer at the boundary. Decide what the smallest legal input is and make the base case catch it *and everything past it*.

Forgetting to return the recursive result. In a value-returning method, writing `factorial(n - 1);` on its own line computes the answer and throws it away. You need `return n * factorial(n - 1);`. The compiler often will not save you.

Exponential recomputation. Naive Fibonacci again: correct, but it recomputes subproblems into oblivion. If your recursion is slow, draw the tree and look for the same node appearing twice. That is your memoization target.

Mutating shared state across calls. Recursion is cleanest when each call works on its own data and *returns* a result. Reaching out to modify a shared or global variable from inside the recursion makes traces lie and bugs multiply. Pass what you need in, return what you computed out.

Blowing the stack on deep but otherwise fine recursion. Even a correct linear recursion can overflow if it goes, say, a million deep, because each frame costs memory. If the depth scales with the input size and the input can be huge, prefer the loop. (Java gives you a clean `StackOverflowError`; C++ tends to just crash, which is nastier to diagnose. See §9.)

9. C++ versus Java: the differences that matter

The recursion *logic* is the same. What differs is the data around it.

- **How arguments are passed.** Java passes everything by value, but for objects the “value” is

a reference, so a recursive method that receives an `int[]` and writes into it changes the caller's array. C++ copies by default, so `std::vector<int> a` as a parameter copies the whole vector *on every call*: correct, but a performance disaster in recursion. Pass containers by reference (`std::vector<int>& a`) or const-reference (`const std::vector<int>& a`) so each call shares the one container instead of cloning it. This is the single most common C++-specific recursion mistake I see.

- **Arrays versus vectors.** Java arrays know their own `length`; C raw arrays do not, so you pass the bounds (`lo`, `hi`) yourself, which we are doing anyway in the range-based recursions above. In modern C++ prefer `std::vector` (it carries its size and is the “secure” choice the C++ book is built around) and pass `lo` and `hi` for the active sub-range.
- **When it goes wrong.** Overrun the stack in Java and you get a catchable `StackOverflowError` with a stack trace. Overrun it in C++ and behavior is undefined, usually a crash or segfault, sometimes silent corruption. Same bug, much friendlier message on the Java side. Test your base case early on the C++ side.
- **Tail calls.** A *tail-recursive* call is one where the recursive call is the very last thing the method does. Some compilers can optimize these into loops, with no growing stack. Do **not** rely on it: the JVM does not guarantee tail-call optimization, and C++ compilers only sometimes do it and only at certain optimization levels. Treat every recursion as if it costs a frame, because portably, it does.

None of this changes the three parts or the trace. It changes how you pass the data and how loudly the program complains when you get the base case wrong.

Last thing. Read these once before class and once after. The first pass makes the live traces make sense; the second pass is where it sticks. Trace at least one example by hand until the calls-down and backtrack-up picture is automatic, because that single skill carries you through the rest of the course. Read the recursion chapter in your textbook too, Prichard and Carrano for Java, Prichard and Henry for C/C++; flip to “recursion” in the table of contents and you will land in the right place. And I mean it about reporting errors: a note that has been wrong for three semesters is only wrong because nobody told me.